

Our Journey in making the Stargazing web application

Alain Izabayo | Jeongho Lee | Jeffry Royce | Kristin Wiseman

Being an amateur astronomer, one of the biggest challenges Jeffry has been facing is identifying the least light polluted locations to visualize celestial objects. When he explained to the team this challenge, we decided to seize this opportunity to help his stargazing community to easily identify what is clearly visible based on their locations. Hence the conception of the idea to create an application that can accurately tell users what is possible to visualize in the sky and where. In this discussion, we explain the app development process; conception, design and implementation. The design and implemented features are examined to evaluate how they help to meet the app's expected usefulness. We also discuss and explain the changes made at each stage, reflect on major technical challenges faced along the way and the proposed possible future improvements.

The web application performance on achieving its usefulness

The primary goal of the app is to enable users to be able to create an account, explore celestial objects and be able to save lists of the objects they observe and potentially share their insights through writing community reports. The application largely achieves these objectives. Moreover the creative component of the app was successfully designed and implemented since users are able to see what is genuinely above their own horizon at the moment they ask. It also allows for judging constellation visibility as a whole rather than one star at a time. In practice this means a user can open the Constellations tab and immediately tell which figures are worth stepping outside for tonight, without any astronomy background or manual lookups. The visibility calculation, the star map, and the ranking all update from the user's own location and time, so the answer the app gives is specific to them.

Comparing the initial and final design schema and ER diagram

Our schema, ER diagram and data sources have not significantly changed from the ones we originally identified in the first part of the project. The only thing we did to the datasets is combine the NGC (deep sky object) catalog with the Hygge database (for stars) to make up our singular CelestialObject table.

We slightly changed the schema of the SavedObject table. Initially, we planned to use a VARCHAR(250) field called ObservedStatus to track whether a CelestialObject has been observed on an Observation List. We chose to remove this field, and replace it with 2 more suitable fields. One is a boolean column called IsObserved, for the user to mark whether or not they've seen that CelestialObject. IsObserved is more suitable than ObservedStatus because whether or not a user has seen something is naturally a boolean. We also added a Notes field, which is a VARCHAR(250) field that allows the user to add some notes about the CelestialObject they saw. These two fields do a better job of keeping concerns separate than the original ObservedStatus field.

Functionalities added and Removed

Among the features we had envisioned but were not able to implement is the ability for users to filter celestial objects visible to them based on a visualization tool. Currently, when the user toggles the "visible tonight" filter, they are only shown objects that are visible to their unaided eyes based on their latitude, longitude, current time, and light pollution measurements near them. Users don't have a way to include more results by selecting that they're using binoculars or a telescope, for example. We chose to remove this proposed feature because we couldn't find a reliable dataset that connected visualization equipment with limiting magnitudes. There are a multitude of reasons for this: the sheer variety of equipment and manufacturers, the effect of aperture size and exposure time on limiting magnitude, and the effects of weather conditions.

Another feature we didn't include is users being able to search by zip code for nearby dark-sky areas. We made this choice so that our app could focus on the most important read operations: displaying information about celestial objects, the user's observation lists, constellations, and community reports. Although finding dark sky areas is relevant to astronomy, we also realized that searching by zip code would be a feature limited to users from the U.S., as some places don't use zip codes, and other countries have different formats for their postal codes.

One feature we added is part of our "creative component". On the "Constellations" tab of our app, the user can see information about all 86 constellations that can be visualized with the unaided eye or commercially available equipment. This information includes a picture of the stars in each constellation, which was created by integrating part of the [D3.js celestial library](#) into our app. The way we implemented this picture is creative and custom to the user, because it relies on the Geolocation Web API to gather the user's device's latitude and longitude, determine the limiting magnitude they experience due to the effects of light pollution, and shows which stars of each constellation are visible. In

this picture, visible stars of the constellation are yellow dots, and invisible stars are grey. The size of each yellow dot in each constellation informs the user of that star's relative brightness: brighter stars have bigger yellow dots. Removing the two features discussed above allowed us to focus on this creative component, which makes our app's feature set both more unique and more relevant to amateur astronomers. Another reason we implemented this feature is because existing astronomy websites and applications provide limited information about constellation visibility as a whole.

This component evolved to take on an educational feature: listing all constellations with links to Wikipedia for additional information. The constellation images illustrate the numerical information we provide about whole-constellation visibility, using a combination of our own data and a 3rd party addition. The stars appear as dots of varying size according to their relative brightness and mapped on an invisible grid according to their celestial coordinate data. Stars not bright enough to pierce the local light pollution are grayed out. The constellation connect-the-dot lines were overlaid using code from d3-Celestial, which combined constellation coordinates (nodes) and connection lines (edges). Credit is given with a NOTICE*.md in `./the-observatory/src/components/`.

The second half of our creative component is the dynamic horizon-line calculator, which determines what is above the user's horizon at the moment they ask. It takes the timestamp and coordinates from the user's device, converts that instant to a Julian Date, derives Local Sidereal Time from it, and applies the standard equatorial-to-horizontal coordinate transformation to every candidate object to produce an altitude in degrees. The formulas were implemented from Meeus' *Astronomical Algorithms* and Duffett-Smith and Zwart's *Practical Astronomy with your Calculator or Spreadsheet*, and they live in a dependency-free Python module so they can be unit-tested in isolation against known reference values.

The role of Advanced database programs

Our advanced database programs help automate common tasks and improve the overall user experience. For example, when a new user is created, a trigger automatically generates a default observation list for them. This allows users to start saving celestial objects immediately without having to manually create a list first. By handling these repetitive tasks inside the database, we reduce the amount of application logic needed, maintain consistent data, and ensure every user begins with the same default setup. In addition, storing this logic in the database means that the behavior is enforced regardless of how the database is accessed, whether through our web application or another client. This reduces the risk of missing or inconsistent data

caused by application-level errors. Our stored procedures also provide standardized ways to perform common operations, such as searching for celestial objects using the Visible Tonight filter and creating community reports, ensuring that these actions follow the same validation rules every time. Overall, these advanced database programs improve reliability, simplify maintenance, and create a smoother experience for users while keeping the database organized and consistent.

Technical challenges encountered

One technical challenge identified by Alain was the difficulty associated with translating an idea into a UML diagram and then a schema. Since none of us had ever designed an app that involves connecting a database to the frontend; imagining how the potential schema should look relative to the user interface became very challenging. In other words it was not straightforward to make a decision on which features should connect to the front end and which ones should simply be coded as front end options either in form of dropdowns or selections rather than being columns in a database. We chose to move forward with having most of the features being detached from the database and displayed on the user interface and then we would make final changes if needed at the implementation phase. The key lesson from here is that it is sometimes too hard to figure out how the final schema or design will look like, thus rather than spending too much time perfecting the design in the beginning it is better to move forward and change the design at implementation if necessary.

A technical challenge that Kristin encountered was underestimating the difficulty of connecting data from the MySQL database to a backend (in our case, Flask/Python), and forwarding that data to the frontend (React/Typescript). This was a challenge for two main reasons: data type management and error handling. Although she understood our database's schema and executed queries in Cloud SQL Studio to validate results, it was a challenge to manage data typing across the layers of the "stack". At a network level, query parameters are always passed as strings, but parameters like latitude and longitude needed to be handled numerically on both the backend for astronomical calculations involving geometry, and on the frontend for compatibility with the proper HTML inputs. Since no one on our team had experience with building a full stack application from scratch, we didn't have an intuition for when to perform type casting or checking. The quality of our database's schema, DDL commands, and constraints meant that Google Cloud MySQL output informative error messages, but we didn't know that the database's messages aren't automatically connected with HTTP status codes and displayed in the browser's console. Essentially, we were unaware of the effort involved in setting up error handling in the backend. Collectively, we learned that the

biggest challenge wasn't database management; it was preventing our database from becoming a data silo.

A technical challenge Jeongho encountered was implementing the Visible Tonight search as a SQL stored procedure. Unlike a typical search that only filters data based on user input, this feature requires translating the application's visibility logic into SQL. The procedure had to combine multiple factors, including the user's location, horizon calculations, and local light pollution, to determine whether a celestial object should be returned in the search results. Converting this multi-step decision process into an efficient SQL query was challenging because the calculations and filtering conditions all had to work together correctly. To overcome this, Jeongho broke the logic into smaller parts, tested each component independently, and verified the results with different locations and observing conditions before combining everything into the final stored procedure. For future teams, Jeongho recommends validating each part of a complex database query separately before integrating it into a single procedure, as this makes debugging much easier and helps identify issues more quickly.

A technical challenge Jeffry encountered was that the data the creative feature needed did not exist. The Constellations tab depends on knowing which stars make up a constellation, but our CelestialObject table only labels each star with the constellation whose region of sky it falls in — roughly 1,977 objects fall within Orion's boundaries, and nothing in the dataset distinguishes the eight bright stars that draw the familiar figure from the background. No public dataset we found supplied that membership in a form that keyed to our catalog. The solution was to derive membership rather than store it, treating a constellation as its brightest stars and using a magnitude cutoff as the working definition. This came with a real tradeoff: at a cutoff of magnitude 3, only 52 of the 88 constellations return any rows at all, 59 have two stars or fewer, and Lyra reduces to Vega alone; At 4.5, 86 of 88 constellations appear with an average of 10.6 stars each, and smaller constellations recovered their full shape.

Potential future improvements

The current app design and implementation is an improvement to the already existing applications. However, in the future we hope to implement the improvements below

- Addition of a feature that enables users to filter CelestialObjects by the tool they plan to use (i.e. telescope). This customization would greatly improve the accuracy and precision of the information derived from the app
- Addition of a map feature that allows users to interactively select a location of their choice and be able to see what can be seen at that location at a given time
- At the moment, the app assumes that all users will be based on planet earth. In future we would explore the possibility of using the app on other planets. This would enable astronauts and other space tourists in future to be able to tell what they can see depending on their location in space

Final division of labor and teamwork

Kristin was the main technical lead of the project. Using her software engineering experience, she helped the team to break down and understand the technical requirements at each stage, tutored us on the best engineering and collaborative design practices. On the app development stage, she implemented the majority of the frontend in React with Typescript, made technical architecture decisions, setup the Flask backend, implemented basic SQL queries that became part of our advanced queries, and assisted with Flask routes for read and create operations. She also made the initial mock user interface in Figma and prepared the git tags for each stage's release.

Jeongho took leadership in laying a foundation for backend development. During the early stages of development, he led discussions on the overall database design, including the creation of the uML diagram and initial relational schema, which served as the starting point for the team's implementation. He also set up the Google Cloud Platform environment and configured the Cloud SQL database, allowing the rest of the team to collaborate using a shared backend infrastructure. As development progressed, other team members refined and expanded the design by modifying the schema and adding new features as the project's requirements evolved. In the final stages of the project, Jeongho worked on the development of the Flask backend by integrating stored procedures, triggers, and constraints. He also tested the backend extensively, including edge cases and error handling, to ensure reliable communication between the application's frontend and backend.

Jeffrey conceived the idea for our web app, and he is the most accomplished astronomy domain expert on the team. Hence he primarily made sure that our idea does not lose

its originality and relevance to the stargazing community. Technically, he was very helpful in designing and reviewing the UML and schema diagrams, conducting the DCNF proof, searching and auditing data sources and quality control of the data. He implemented equations for calculating complex astronomical geometry like local sidereal time, providing the basis of the app's functionality that determines which stars are visible to a user based on their position on the planet. He implemented the majority of our advanced queries as well as a stored procedure and transaction. In the final stages of the project he helped on both the front end and backend components of the project, including our "creative component" which uses part of the [D3.js celestial library](#) to visualize 86 different constellations, displaying which of their stars are visible for a user's location in an SVG.

Alain handled most of the logistical components of the project. Ensuring that all the requirements of the assignments were met and submissions were on time and work was as proportionally shared among the teammates as possible. He jumped in to help at all stages, for example helping with the design of the schema, indexing databases and on some components of the backend in the final stage of the project. He was the main architect of this final report and the project video.

Overall, the cycle of work involved Alain identifying project assignment requirements and deadlines and then proposing meeting times. Kristin broke down the technical requirements at each stage and invited members to volunteer for identified tasks at each stage. Jeffry and Jeongho ensured that the way the tasks are broken is within the expected usefulness targets and weighed on the technical depth of each task. And then we distributed the tasks amongst ourselves and then reassigned tasks depending on the workload.

This dynamic and flexible approach helped us to easily change our task assignments depending on the situation at hand - hence successfully designing the stargazing web application.